



LAPORAN PRAKTIKUM

Pengembangan Aplikasi Web II

Disusun oleh:

Nama : Aldi Wijaya

NIM : 2428240162

Kelas : SI5C

PROGRAM STUDI SISTEM INFORMASI
UNIVERSITAS MULTI DATA PALEMBANG
TAHUN AKADEMIK 2025/2026

Identitas Praktikum

Nama Mahasiswa	Aldi Wijaya
NPM	2428240162
Kelas	SI5C
Pertemuan Ke-	2
Topik / Judul Praktikum	Full rest API
Tanggal Pelaksanaan	
Dosen Pengampu	<u>Nur Rachmat, M.Kom</u>

I. Tujuan Praktikum

Tuliskan tujuan praktikum sesuai indikator capaian pembelajaran pada pertemuan terkait (lihat slide materi bagian “Capaian Pembelajaran”).

Sub-CPMK 4 Mahasiswa mampu membuat backend RESTful API menggunakan Express.js. (C3)

II. Dasar Teori

Uraikan secara singkat konsep/teori yang mendasari praktikum ini (maksimal 1 halaman). Sertakan sumber rujukan bila mengutip dari pustaka.

1. Node.js

Node.js merupakan *runtime* JavaScript yang menggunakan mesin V8 dan memungkinkan JavaScript dijalankan

an di luar browser, khususnya pada sisi server. Node.js menjadi dasar dalam pengembangan aplikasi menggunakan Express.js serta mendukung ekosistem *package* melalui NPM.

2. NPM (Node Package Manager)

NPM merupakan pengelola *package* yang terpasang bersama Node.js. NPM digunakan untuk menginstal, memperbarui, dan menghapus *dependency* yang dibutuhkan dalam suatu proyek. Daftar *dependency* dicatat dalam `package.json`, sedangkan `package-lock.json` digunakan untuk mengunci versi *package* agar konsisten pada lingkungan pengembangan.

3. Express.js

Express.js adalah *framework* minimalis untuk Node.js yang mempermudah pembuatan server web dan API. Express.js menyediakan fitur seperti *routing* untuk memetakan URL dan HTTP method ke fungsi penanganan, serta *middleware* untuk memproses *request* sebelum diteruskan ke *handler*.

4. RESTful API

REST (*Representational State Transfer*) merupakan gaya arsitektur untuk merancang API berbasis *resource* yang diakses menggunakan HTTP. Dalam RESTful API, setiap data direpresentasikan sebagai *resource* melalui URL, setiap *request* bersifat independen (*stateless*), dan HTTP method digunakan sesuai operasi yang dilakukan. Operasi CRUD umumnya dipetakan ke GET, POST, PUT, dan DELETE, dengan JSON sebagai format pertukaran data.

5. Routing

Routing adalah mekanisme pada Express.js yang menentukan bagaimana aplikasi merespons *request* berdasarkan URL dan HTTP method tertentu. Express.js menyediakan metode seperti `app.get()`, `app.post()`, `app.put()`, dan `app.delete()` untuk menangani operasi pada endpoint API.

6. Request dan Response

Dalam Express.js, objek request (`req`) digunakan untuk memperoleh data yang dikirim oleh client, seperti `req.params` untuk parameter pada URL, `req.query` untuk *query string*, dan `req.body` untuk data pada *request body*. Sementara itu, objek response (`res`) digunakan server untuk mengirimkan hasil kepada client, seperti melalui `res.json()` dan `res.status()`.

7. Middleware `express.json()`

Middleware `express.json()` digunakan agar Express.js dapat membaca data JSON yang dikirimkan pada *request body*. Middleware ini perlu didaftarkan sebelum *route* yang membutuhkan akses terhadap `req.body`.

8. HTTP Method dan CRUD

Dalam RESTful API, operasi CRUD dapat diterapkan menggunakan HTTP method. GET digunakan untuk membaca data, POST untuk menambahkan data, PUT untuk memperbarui data, dan DELETE untuk menghapus data. Setiap operasi juga dapat menghasilkan kode status HTTP yang sesuai, seperti 200 OK, 201 Created, 404 Not Found, dan 204 No Content.

9. Format Response JSON

Response pada RESTful API umumnya menggunakan format JSON. Response yang

berhasil menggunakan kode status kelompok 2xx dan dapat menyertakan data pada *response body*, sedangkan kesalahan menggunakan kode status 4xx atau 5xx disertai pesan yang menjelaskan kesalahan. Konsistensi format response membantu client dalam menggunakan API.

III. Alat dan Bahan

Sebutkan perangkat lunak/perkakas yang digunakan selama praktikum.

No.	Nama Alat / Software	Versi / Keterangan
1	Node.js	Versi 24.19
2	NVM	Mengelola versi Node.js
3	Visual Studio Code	Versi Terbaru
4	NPM	Package manager untuk mengelola dependensi
5	CMD/Command Promt	Menjalankan perintah instalasi dan server
6	Express.js	Framework Node.js untuk membuat server dan API

IV. Langkah Kerja

Jelaskan langkah-langkah yang dilakukan selama praktikum secara berurutan.

1. **Menyiapkan environment** dengan memastikan Node.js dan NVM telah terpasang, kemudian mengecek versi Node.js yang digunakan.

2. **Membuat project Node.js** dengan membuat folder project dan menjalankan npm init -y.

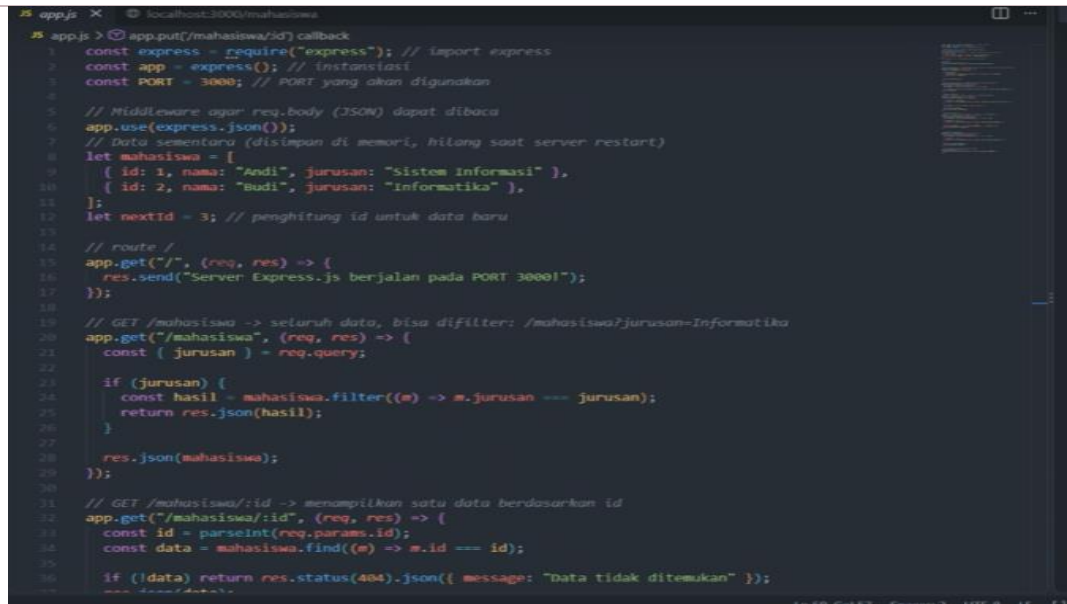
3. **Menginstal Express.js** menggunakan perintah npm install express sebagai framework untuk membangun RESTful API.

4. **Membuat dan menjalankan server** dengan membuat file app.js, kemudian menjalankannya menggunakan node app.js.

5. **Mengembangkan dan menguji API** dengan membuat routing serta operasi CRUD menggunakan method GET, POST, PUT, dan DELETE, kemudian menguji setiap endpoint untuk memastikan API berjalan sesuai kebutuhan.

V. Hasil dan Pembahasan

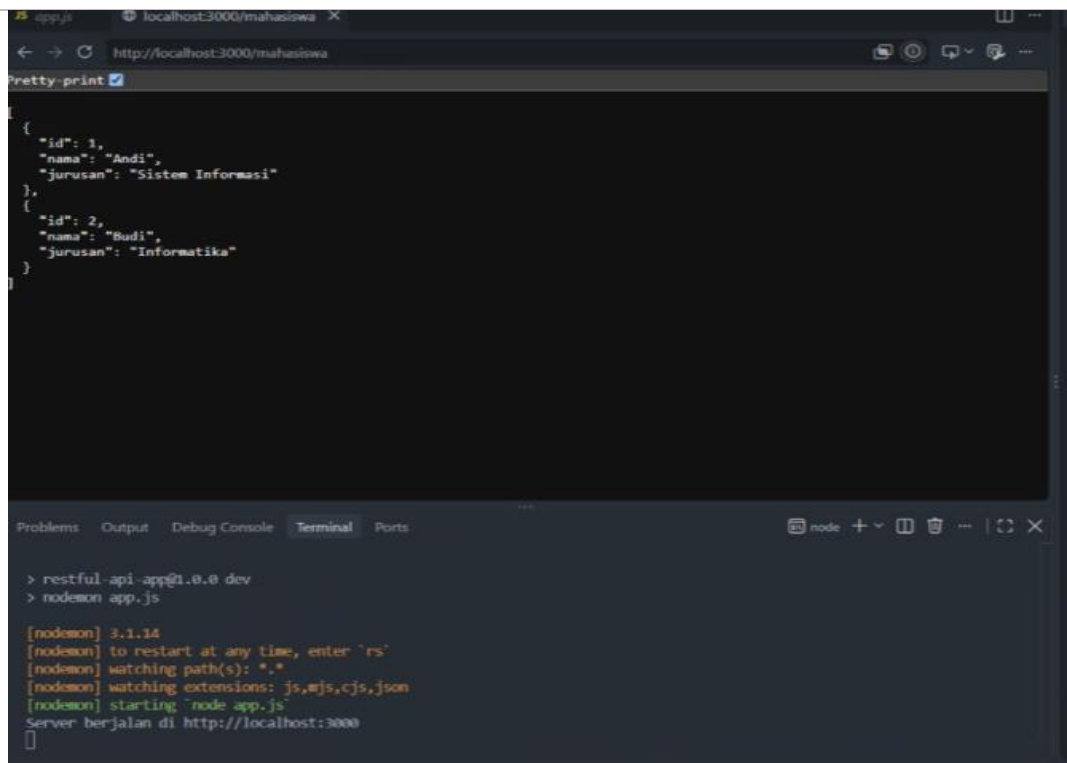
Sertakan tangkapan layar (screenshot) hasil praktikum, disertai penjelasan/pembahasan pada setiap hasil.



```
1 app.js > app.put('/mahasiswa/:id') callback
2 const express = require("express"); // Import express
3 const app = express(); // Instansi
4 const PORT = 3000; // PORT yang akan digunakan
5
6 // Middleware agar req.body (JSON) dapat dibaca
7 app.use(express.json());
8 // Data sementara (disimpan di memori, hilang saat server restart)
9 let mahasiswa = [
10   { id: 1, nama: "Andi", jurusan: "Sistem Informasi" },
11   { id: 2, nama: "Budi", jurusan: "Informatika" },
12 ];
13 let nextid = 3; // penghitung id untuk data baru
14
15 // route /
16 app.get("/", (req, res) => {
17   res.send("Server Express.js berjalan pada PORT 3000!");
18 });
19
20 // GET /mahasiswa -> seluruh data, bisa difilter: /mahasiswa?jurusan=Informatika
21 app.get("/mahasiswa", (req, res) => {
22   const { jurusan } = req.query;
23   if (jurusan) {
24     const hasil = mahasiswa.filter((m) => m.jurusan === jurusan);
25     return res.json(hasil);
26   }
27   res.json(mahasiswa);
28 });
29
30 // GET /mahasiswa/:id -> menampilkan satu data berdasarkan id
31 app.get("/mahasiswa/:id", (req, res) => {
32   const id = parseInt(req.params.id);
33   const data = mahasiswa.find((m) => m.id === id);
34   if (!data) return res.status(404).json({ message: "Data tidak ditemukan" });
35   res.json(data);
36 });
```

[Tempelkan tangkapan layar (screenshot) di sini]

Gambar 1. Proses konfigurasi



The screenshot shows a web browser at `http://localhost:3000/mahasiswa` displaying a JSON array of two students: Andi (Sistem Informasi) and Budi (Informatika). Below the browser, a terminal window shows the command `node app.js` being executed, with output indicating the server is running on `http://localhost:3000`.

[Tempelkan tangkapan layar (screenshot) di sini]

Gambar 2. Hasil outputnya

Pembahasan:

VI. Kendala dan Solusi

Optional - isi apabila terdapat kendala teknis selama praktikum.

No.	Kendala	Solusi
1		
2		

VII. Kesimpulan

Simpulkan hasil praktikum dikaitkan dengan tujuan pada bagian I.

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa Node.js dan Express.js dapat digunakan untuk membuat dan menjalankan server sederhana. Pada praktikum ini, project Express.js berhasil dijalankan melalui localhost:3000 sehingga dapat menjadi dasar untuk mempelajari pembuatan RESTful API pada tahap selanjutnya.

Daftar Pustaka

Tuliskan sumber rujukan yang digunakan (format APA 7th edition).

Lampiran

Tautan Repository GitHub:

[Alwjy-ix/si5c_latihan_restful_aldiwjy: restfull-api-app](#)

Tautan Deployment (jika ada):

[si5c-latihan-restful-aldiwjy.vercel.app](#)